الجمهورية العربية السورية

الجامعة السورية الخاصة

كلية هندسة الذكاء الاصطناعي

Agentic AI Data Analyst

semester project prepared to obtain the degree of B.Sc. in the Department of Artificial Intelligence Engineering and Data Science

Authors :

مجد هيثم الحميدان 4220289

خليل يحيى الهلال 4220052

Supervised by:

الدكتورة ميساء ابو قاسم

المهندس احمد شيخة

العام الدراسي 2025–2026

SUPERVISOR CERTIFICATION

I certify that the preparation of this project, entitled **Agentic AI Data Analyst** prepared byMajd Homedan, Khalil Yahyawas carried out under my supervision at the Faculty of Artificial Intelligence Engineering in partial fulfillment of the requirements for the degree of **B.Sc.** in the Department of Artificial Intelligence Engineering and Data Science.

Name : Maysaa Abu Qasim

date :

Signature :

كلمة شكر وتقدير و اهداء

جرى هذا المشروع في كلية هندسة الذكاء الصنعي، الجامعة السورية الخاصة. نود أن نشكر المشرفين على هذا المشروع الدكتورة ميساء ابو قاسم و المهندس أحمد شيخة

كما نتقدم بالشكر والثناء إلى الكادر التدريسي في كلية هندسة الذكاء الصنعي بالإضافة الى زملاء دراستنا و أصدقائنا ولكل من مد لنا يد العون لإنجاز هذا البحث.

كما نود أن نعرب عن امتناننا لأهالينا الذين ساندونا في سنوات دراستنا ...

الملخص

1. تعريف المسألة وأهميتها والتحديات المرتبطة بها

تتجلى مشكلة البحث في الفجوة المتزايدة بين حجم البيانات الضخمة المتولدة يومياً وبين قدرة المحللين البشريين على استخراج رؤى دقيقة وسريعة منها، بالإضافة إلى تعقيد أدوات تحليل البيانات التقليدية التي تتطلب مهارات برمجية عالية. تكمن أهمية المشروع في أتمتة دورة حياة تحليل البيانات (Data Analysis Lifecycle) لتمكين المستخدمين غير التقنيين من التفاعل مع البيانات. أما التحديات المرتبطة بها فتشمل ضمان دقة الاستنتاجات الإحصائية، التعامل مع البيانات غير المهيكلة، وإدارة الوكيل بشكل صحيح لتجنب الهلوسة التقنية في النتائج.

2. توصيف الحل المقترح بإيجاز، ونقاط قوته

يقوم هذا المشروع ببناء نظام وكيل ذكاء صنعي (Single Agentic AI System) يعتمد في عمله على منهجية ال ReAct وهي تقنية تسمح للوكيل بدمج التفكير المنطقي مع اتخاذ إجراءات فعلية في بيئة العمل بشكل متزامن للوصول للهدف ،وتكمن القيمة المضافة للنظام في قدرة هذا الوكيل على تولي كامل دورة حياة تحليل البيانات ،تبدأ من فهم الاستعلامات وصولا الى استخراج النتائج حيث يقوم الوكيل بتنظيف البيانات والتحليلات الإحصائية و توليد التصورات (Visualizations) وتقديم تقرير نهائي يحتوي على الهدف والشرح بطريقة مفهومة للمستخدم، مما يوفر الوقت والجهد بشكل كبير جدا على المستخدم، يبرر هذا التصميم كفاءة النظام في تقديم حلول متكاملة مع تقليل الأخطاء البشرية و دون الحاجة لتدخل بشري مستمر .

3. استعرض النتائج

Abstract

1. Problem definition, its importance, and associated challenges

The research problem is manifested in the growing gap between the massive volume of data generated daily and the ability of human analysts to extract accurate and rapid insights from it, in addition to the complexity of traditional data analysis tools that require advanced programming skills. The importance of the project lies in automating the Data Analysis Lifecycle to enable non-technical users to interact with data. The associated challenges include ensuring the accuracy of statistical conclusions, dealing with unstructured data, and properly managing the agent to avoid technical hallucinations in the results.

2. A brief description of the proposed solution, and its strengths

This project builds a Single Agentic AI System that operates based on the ReAct methodology, a technique that enables the agent to combine logical reasoning with taking real actions in the work environment simultaneously to achieve the goal. The added value of the system lies in the agent's ability to handle the entire data analysis lifecycle, starting from understanding queries all the way to extracting results. The agent performs data cleaning, statistical analysis,

generates visualizations, and delivers a final report that includes the objective and explanation in a way that is understandable to the user. This significantly saves time and effort for the user. This design justifies the system’s efficiency in providing integrated solutions while reducing human errors and eliminating the need for continuous human intervention.

3. Display results

فهرس المحتويات

Index of Tables

Index of Figures

Glossary of Terms

1.Chapter One: Introduction to the Research, Scientific Problem of the Research, Objective of the Research

1.1 Introduction to the Research

In 2023, the term "Agentic AI" emerged following the surge of Generative AI. However, the concept remained largely theoretical until 2025, which marked the actual breakthrough for this field. It transitioned from abstract models into practical tools that are fundamentally transforming work environments, acting as active partners capable of executing complex, multi-step tasks autonomously.

In the context of Artificial Intelligence, an "Agent" is defined as a sophisticated software entity capable of perceiving its environment, reasoning about tasks, and taking necessary actions to achieve specific goals without continuous human intervention. In this project, the "Agent" serves as the system's "brain." It does not merely follow a fixed set of rules; instead, it employs Orchestration Logic and reasoning loops (such as ReAct: Reasoning and Acting) to determine the appropriate tool for use, self-correct its errors, and interpret complex instructions.

Data Analysis is a systematic process of inspecting, cleansing, transforming, and modeling data to discover useful information and support decision-making. Traditionally, this process requires a human analyst to manually write code (e.g., Python or SQL), handle missing values, and generate visualizations.

The core of this project lies in integrating the Agentic AI workflow with data analysis pipelines through a Tool-based system. By relying on pre-defined, manually developed tools rather than autonomous code generation, the system significantly enhances reliability and eliminates the risks of "AI hallucinations" in critical data transformations. In this framework, the Agent handles the entire lifecycle of the process—from data ingestion and cleansing to generating statistical visualizations and synthesizing a final textual report presented in a human-readable format.

1.2 The scientific problem of the research

In the current era, the world is witnessing a massive surge in the volume and diversity of information, leading to Data Inflation & Big Data. This has created complex technical and practical challenges in data processing and analysis. Traditionally, data science teams face a fundamental issue: every new dataset arrives with a unique structure, forcing human analysts to write entirely new Scripts or perform radical modifications to existing code. Scientific studies in this field highlight the "80/20 Rule," which states that data analysts historically spend 80% of their time and effort on the tedious tasks of data preparation and cleaning, leaving only 20% for actual analysis and insight generation. This excessive reliance on Manual Coding creates a significant Bottleneck, where Time Inefficiency hinders the speed of decision-making, making the process slow and Unscalable in the face of continuous, real-time data streams.

The **"Agentic AI Data Analyst"** project serves as a radical and effective solution to this scientific problem by shifting the focus from repetitive manual programming to **Intelligent Automation**. By relying on an autonomous agent equipped with pre-defined tools, the system eliminates the need to write new code for every file; the Agent is capable of dynamically understanding the data structure and immediately applying the appropriate analytical tool. A key strength of this system is its **Adaptability**, as it remains unaffected by changes in column names, relying instead on the **Semantic Understanding** of variables and their context rather than rigid programmatic identifiers.

This approach significantly minimizes **Human Errors** (such as logic or syntax errors caused by fatigue) and ensures high-accuracy results. Most importantly, the project contributes to achieving **Data Democratization**. It breaks down the technical barrier, allowing non-specialized users—such as managers and business owners—to interact with their data and extract professional reports without any prior programming knowledge. Consequently, this transforms data from a complex technical asset into a powerful, accessible knowledge tool for everyone.

1.3 Project Objective

The central objective of this project is to develop an intelligent system capable of automating the entire data analysis lifecycle, from raw data ingestion to the synthesis of final results. The project seeks to establish a functional model where the AI Agent acts as an "Autonomous Analyst," responsible for orchestrating various programming tools without human intervention. The goal transcends mere code execution; it aims to build a system possessed of **Procedural Awareness**, enabling it to evaluate data quality and make logical decisions regarding the most appropriate analytical methods for each specific case. Through this automation, we aim to transform data analysis from a complex, labor-intensive process into a "seamless" and highly efficient workflow capable of handling diverse data structures with total flexibility, setting a new standard for intelligent system interaction with large datasets.

In addition to comprehensive automation, the project aims to achieve several strategic objectives tailored to modern business environments:

- Operational Cost Reduction: A fundamental pillar of this project is enhancing cost efficiency. The system aims to minimize reliance on intensive human resources for routine and repetitive tasks. By automating the labor-heavy stages of data cleaning and preparation, the system significantly lowers the "cost-per-analysis," allowing organizations to reallocate their human capital toward more creative and high-level strategic tasks.
- Scalability & Portability: The project seeks to build a resilient framework compatible with any data source (such as Excel, CSV, or SQL databases) without requiring system reconfiguration. This ensures sustained performance regardless of increasing data volume or the complexity of its sources.
- Deep Insight Generation & Real-time Decision Support: Beyond surface-level understanding, the system is designed to identify complex correlations between variables to extract intelligent insights. This supports real-time decision-making, allowing stakeholders to

obtain precise answers during live meetings without the traditional days-long wait for manual reports.

- Data Democratization & Automated Documentation: The project aims to dismantle the programming barrier for non-technical users while ensuring Transparency. The system provides "Automated Documentation," where the Agent explains the methodology it followed to reach its conclusions, thereby fostering user trust in the accuracy and credibility of the generated reports.

2.Chapter Two: Reference Studies

2.1 First Reference Study :

ReAct: Synergizing Reasoning and Acting in Language Models (2023)

In 2022, the research field was grappling with two parallel, yet unintegrated, capabilities in large language models. The first was chain-of-thought reasoning, which generates step-by-step thinking but relies entirely on the model's memory, opening the door to hallucinations and the spread of errors through the thought process. The second was act-only reasoning, which enables the model to interact with external sources but operates without planning or prior thought, making it vulnerable to unforeseen outcomes. The separation of these two capabilities posed a significant obstacle to building reliable agents capable of performing .multi-step tasks in a constantly evolving real world

ReAct emerged to declare that this separation is not a structural necessity but rather a design choice that can be bypassed. The paper provides extensive empirical evidence that integrating thinking and action into a single, interconnected loop improves performance, reduces errors, and produces understandable and verifiable solution .pathways

This paper proposes a framework that enables a language model to simultaneously generate both verbal reasoning traces and task-specific actions. The name ReAct stands for Reasoning and Acting. The core principle is that reasoning traces help the model deduce, track, update, and handle exceptions, while actions allow it to connect with external sources to gather additional information that feeds into reasoning. This two-way interaction creates a continuous feedback loop: reasoning improves action, and action feeds .into reasoning

The system relies on a sequential, iterative structure centered around three components that repeat in each cycle. The first component is THOUGHT, or Idea, which is natural text generated by the model describing its assessment of the current situation and its plan for the next step. The second component is ACTION, or Action, which is the invocation of an external tool with specific inputs, such as an encyclopedia search, a database query, or the execution of code. The third component is OBSERVATION, or Observation, which is the result of the tool's execution and is added to the model's context to build upon for its next idea. This three-part loop repeats until the model reaches FINAL ANSWER or exceeds the maximum number of .allowed steps

What distinguishes this design is that thought and action share the same generative space within the model, making the entire process readable and verifiable. The model doesn't switch between two states but generates a continuous text punctuated by natural thought and action. This transparency enables humans to intervene and correct the course if the model deviates, as documented in the .paper's "Human-in-the-loop correction" experiments

The paper tested ReAct in four environments representing different types of interactive tasks. In the HotPotQA environment, which answers multi-step questions, the Wikipedia API was used as a single tool enabling keyword searches or access to a specific page. In the FEVER fact-checking environment, the same tool was used, but with a task to verify a claim. In the ALFWorld interactive text environment, which simulates a home environment, text verbs such as "go to," "pick up," and "use" were used. In the WebShop environment, verbs for navigating an e-commerce website, such as "search," "click," and "buy," were used. The common thread across these environments is that ReAct works with any callable tool that returns a result, without .limitations on the tool type or domain

Although the paper doesn't directly address data analysis, the framework naturally applies to this area. When a user enters an analytical request, the language model receives the request along with a description of the tools available in the System Prompt. In the first THOUGHT, the model assesses what it needs to understand and decides to read the file first. In the first ACTION, it calls the DataReaderTool and passes the file path. In the first OBSERVATION, it receives a summary of the data format, column types, and missing values. In the second THOUGHT, it evaluates its findings and plans the next step—cleaning, analysis, or graphing—based on its results. This loop continues until the .complete analysis and final report are available

On the HotPotQA benchmark, ReAct significantly outperformed Chain-of-Thought alone by reducing hallucination errors, as the Wikipedia API allows for the verification of each piece of information instead of relying on model memory. On the ALFWorld benchmark, it achieved a success rate exceeding 71%, compared to 45% for its top competitors—an absolute improvement of 34%. On the WebShop benchmark, it achieved an absolute improvement of 10% compared to imitation and reinforcement learning methods, and this was based on just two learning models. The most important finding is that combining thought and action produces solution paths that are more human-interpretable, diagnostic, and correctable—a characteristic that is just as important as numerical .accuracy

What did the study add to our research

The thought-action-observation loop described in this paper is the heart of the data analytics client proposed in our project. Citing it justifies the decision to use ReAct instead of a simple chain-of-thought approach or a direct tool-calling approach, and demonstrates that this framework produces more reliable and transparent clients. The transparency that ReAct provides will show the user in our project every step .of the thought and action process

2.2 Second Reference Study :

InfiAgent-DABench: Evaluating Agents on Data Analysis Tasks (2024)

At the beginning of 2024, a clear gap existed in the research landscape: there was no specialized assessment standard that measured LLM clients' ability to comprehensively and objectively analyze data from end to end. Existing standards such as HumanEval and DS1000 measured code generation in small, isolated tasks and lacked assessment of sequential planning, the ability to call a real execution environment, the ability to handle diverse and potentially dirty real-world data files, and the ability to extract meaningful final answers. The deeper gap lay in the nature of the data analysis questions themselves, which were open-ended and difficult to evaluate automatically without human oversight, thus hindering .comprehensive and comparative assessment

This paper presents two complementary contributions. The first is the DAEval dataset, which in its updated version contains 603 analytical questions extracted from 124 CSV files collected from real GitHub repositories. These files cover diverse fields, including commerce, sports, crime, and more. The questions are comprehensive, covering concepts such as filtering, clustering, meta-statistics, correlation analysis, trend detection, handling missing values, and timescale analysis. Format-Prompting technology was used to convert each open-ended question into a closed format, enabling automatic comparison with a reference answer .without human intervention in each case

The second is a complete client framework that enables any language model to run as a data analysis client. The framework uses ReAct as the default baseline and provides a complete infrastructure that includes API calling for GPT-4, GPT-3.5, Cloud, and other models, local inference for open-source models via vLLM for those who want to work on their own servers, a Python Sandbox environment built on Docker for secure and isolated execution, and a Streamlet-based front-end for visual interaction with the .system

The system consists of three distinct layers that work in a coordinated manner. The first layer is the Streamlet interface, which allows the user to upload a CSV file, input data via a dialog box, and track the client's progress step by step. The second layer is the client layer, built on ReAct Pipeline. This logical layer receives the file and query and manages the entire inference loop. It includes formatting logic that reformats the answer into an evaluable format. The third layer is the Python Sandbox, built on Docker. This isolated environment receives the generated code, executes .it, and returns the result to the second layer

When a CSV file is uploaded to the system, the cycle begins with the language model receiving the file along with a query and a description of the available tools. The first `Thought` command generates a statement describing its intention to read the file and understand its structure. The first `Action` command generates Python code that calls `pd.read_csv()` with the file path. This code is sent to the Docker Sandbox, where it executes safely in an isolated environment. The result is returned as a text file called `Observation`. From this observation, the client learns the number of rows and .columns, their names, data types, and statistics

If the client in the next THOUGHT finds missing values, as instructed by OBSERVATION, it generates an ACTION that calls df.fillna() with the appropriate value. If it needs grouping, it calls df.groupby(). If it needs a plot, it calls matplotlib.pyplot or seaborn. All these calls are executed in a sandbox and return their results as OBSERVATION. What makes this cycle particularly suitable for our project is that it doesn't assume a specific file format: the client explores the file itself in the first step and then adapts all subsequent .steps

The results were based on the evaluation of 34 language models on DAEval using zero-shot prompting. GPT-4 achieved the highest score among commercial models at 78.99%, while GPT-3.5 achieved 67.59%. Among open-source models, Qwen-72B-Chat led with 59.92%. The specialized model DAAgent-34B, developed and configured by the team on the DAInstruct suite, surpassed GPT-3.5 by 3.89%, demonstrating that customization in data analysis improves performance even with smaller models than commercial competitors. The key takeaway is that data analysis tasks are challenging for current LLM clients, and .even the best models still fall short of human performance

?What did the study add to our research

Three elements of our project are directly inspired by this research. The first is the Python Sandbox, where we will adopt the principle of an isolated execution environment for the generated code, and our use of subprocess achieves the same idea with a simplification suitable for an academic development environment. The second is the Streamlet interface, where the system the user will interact with is inspired by this framework, allowing for file uploading, goal input, and real-time progress monitoring. The third is the evaluation methodology, which compares the client's output with reference responses, representing the same principle we will use to compare our client's insights with those of a .human analyst

2.3 Third Reference Study:

Data Interpreter: An LLM Agent For Data Science(2025)

Despite their success in single, isolated data science tasks, LLM systems suffered from three limitations when required to perform a complete workflow. The first limitation was their inability to adapt dynamically: when data changed mid-analysis or unexpected anomalies were detected, systems based on a fixed plan could not replan. The second limitation was poor tool integration, as the invocation of specialized analysis libraries was random and unstructured. The third limitation was their failure to learn from mistakes and leverage the experience gained during the analysis .session

The paper presents three techniques that together constitute the system's original intellectual contribution. The first technique is dynamic planning with hierarchical structures of graphical data. Instead of a fixed, one-time-generated plan, the large task is broken down into a network of nodes, where each node represents a specific sub-task. The fundamental innovation is that this network is not static but rather dynamic: the results of any node can reshape subsequent nodes by adding, modifying, or rearranging them. The second technique is dynamic tool integration, which enables the system to import specialized tools during execution based on its discoveries, thus reducing reliance on pre-programmed knowledge. The third technique is experience logging, which documents execution successes and failures so the system can refer back to them when .encountering similar situations

The system consists of four main components. The Hierarchical Graph Engine is the core component that receives the complete task, generates the initial plan, monitors execution, and dynamically updates the network. The Code Executor executes the generated code for each node, handles errors, and runs the retry mechanism. The Tool Library is a library of tools with descriptions that help the model select the appropriate one, including Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, and StatsModels. The Experience Manager records the execution results and .allows the model to retrieve them to avoid recurring errors

When a user inputs a complete analytical task, Graph Engine generates an initial plan that breaks it down into sequential nodes. The first node executes `pd.read_csv()`, then `df.info()`, `df.describe()`, and `df.head()` to obtain a comprehensive view of the data. The second node examines `df.isnull().sum()`, handles missing values, validates data types, and removes duplicates. The third node performs statistical analysis, including correlation calculations, data grouping, and descriptive statistics. The fourth node generates the appropriate graphs. The fifth node compiles .the final report

The most important aspect is dynamism: if the second node detects a problem with the date column formatting, for example, Graph Engine automatically adds node 2.1 to address this issue before moving on to the third node. This automatic adaptation is what distinguishes Data Interpreter from fixed-plan systems that fail when faced with .unexpected challenges

Data Interpreter was compared to the best open-source systems. On machine learning tasks compared to OpenML, it improved from 0.86 to 0.95, a 10.5% increase. On the MATH benchmark for mathematical problems, it improved by 26%. On open-ended tasks, it achieved an exceptional 112% improvement, reflecting the true value of dynamic planning in tasks without a predefined path. On InfiAgent-DABench, it increased the accuracy of answers .from 75.9% to 94.9% compared to the base system

?What did the study add to our research

The sequence of the five tools in our project is directly inspired by the principle of decomposing tasks into nodes. The core idea—teaching the client to break down data analysis tasks into logically sequential steps rather than responding to requests as a single unit—is what will be coded in our client's System Prompt based on this paper. Similarly, the concept of proactive subtasks that initiate upon detecting anomalies reflects the same principle of .dynamically adding nodes in Data Interpreter

2.4 Fourth Reference Study :

DatawiseAgent: A Notebook-Centric LLM Agent Framework for Adaptive and Robust Data Science Automation (2025)

DatawiseAgent starts with a simple question: How do human data scientists actually work? They work in Jupyter Notebooks, writing a cell of code, executing it, observing its results, annotating their observations with a markdown cell, and then writing the next cell. This incremental, exploratory approach teaches data scientists how to interact with data naturally. The problem is that existing systems suffer from a narrow scope of tasks, each specializing in a single type; limited generalization across models, which degrades when using models weaker than GPT-4; and an over-reliance on .elite models, which restricts practical deployment

The first component is Unified Interaction Representation. Everything in DatawiseAgent is expressed in a single, unified format: a sequence of notebook cells alternating between text markdown cells and executable code cells. User questions, customer responses, and plans are written in markdown cells. The executed code comes in code cells. The results of the execution appear as the output of these cells. This unification reduces cognitive friction and makes the context clear to the model at all times, enabling smaller .models to perform effectively

The second component is the FST-Based Multi-Stage Architecture, which governs workflow across four structured .functional stages

The first stage, DFS-Like Planning, builds a comprehensive analytical plan using a deep search strategy that systematically explores the solution space before writing any code. The second stage, Incremental Execution, executes the plan cell by cell, utilizing immediate feedback for each cell before moving to the next, directly analogous to the exploratory method used by data scientists. The third stage, Self-Debugging, is activated automatically when a code cell fails to execute. The client analyzes the error message, diagnoses the cause, and generates corrected code, as illustrated in the example in the paper: when KeyError occurs in WINDSPEED, it is corrected to Wind_Speed and the execution is restarted. The fourth stage, Post-Filtering, removes redundant or failing cells to produce a clean, final .notebook that is reviewable and verifiable

DatawiseAgent runs on top of the Jupyter Notebook environment with Docker Sandbox for secure execution. It supports OpenAI's GPT-4o and GPT-4o-mini, and a list of Qwen 2.5 models in 7B, 14B, 32B, and 72B sizes via vLLM. Analysis libraries include Pandas, NumPy, SciPy, Matplotlib, and Seaborn, as well as the Vision Tool for visual verification .tasks

When an analytical request is submitted, DatawiseAgent begins with a markdown cell outlining the complete plan. This is followed by a code cell that imports libraries, reads the data, and executes it in Docker. Crucially, the client treats the data like an explorer, not a rigid plan executor: upon first reading a file, it generates exploratory questions about its structure and then adapts subsequent steps based on the answers. This means that data cleanup doesn't begin with a fixed list of actions but rather with understanding the .actual problems within the data

On InfiAgent-DABench, DatawiseAgent with GPT-4o-mini outperformed all comparable systems, including AutoGen, ReAct, and TaskWeaver, achieving an ABQ score of 94.93%. On MatplotBench, it achieved a completion rate exceeding 90% across all tested model settings, surpassing even the dedicated MatplotAgent. On DSBench, it demonstrated outstanding performance, even with smaller models, exhibiting a graceful degradation that underscores the robustness of its design. Most importantly, DatawiseAgent is the first system to achieve SOTA on three .different types of data science tasks simultaneously

?What did the study add to our research

The order in which the five tools are called in our project is read, then clean, then parse, then plot, then explain. This sequence is empirically supported by DatawiseAgent's results, which have proven its effectiveness. A self-correction mechanism will be implemented in our project as a retry mechanism when the code fails. Python Sandbox, as a principle of safe execution, is supported by the actual .implementation of DatawiseAgent with Docker

2.5 Fifth Reference Study:

AutoDCWorkflow: LLM-based Data Cleaning Workflow Auto-Generation and Benchmark (2025)

Data cleaning consumes over 80% of data scientists' time, according to documented statistics in the research literature. It is also the least engaging and most error-prone phase of the process. Despite the existence of specialized tools like OpenRefine, the decision to select the appropriate data remains manual: the user decides which process to apply, in what order, and on which columns. This decision requires a deep understanding of the nature of the data, the purpose of the analysis, and the connections between them. The problem deepens when we realize that there is no universal cleaning workflow that works for all situations: what cleans .sales data differs from what cleans healthcare data

The key feature of AutoDCWorkflow is its purpose-driven cleaning principle. Instead of randomly cleaning the entire table, the system takes two inputs: the raw table and the purpose of the analysis, expressed as a query. The goal is to produce a minimally clean table that provides the minimum quality necessary to achieve that specific purpose without wasting resources cleaning columns irrelevant to the .required analysis

The first component, Select Target Columns, receives the table description and query text and identifies the set of columns directly relevant to the desired analysis. For example, if the query is for average price by category, it will only select the Price and Category target columns, even if the table contains twenty other columns. This selection reduces the complexity of the cleanup and focuses .resources

The second component, Inspect Column Quality, performs a comprehensive inspection of the quality of each target column and produces a Data Quality Report that classifies problems into three types that the paper explicitly addresses: duplicates, which affect the accuracy of the assemblies; missing values, which hinder statistical analysis; and format inconsistencies, such as dates written in multiple formats or .numbers embedded in texts

The third component, Generate Operations and Arguments, generates a series of appropriate cleanup operations based on the quality report, each with its own specific parameters. The paper identifies six basic operations that address common cleanup issues: removing duplicate rows, filling in missing values with a specific, adjacent, or most frequent value, standardizing date formats, correcting data types, and deleting rows whose values exceed a certain threshold. The .cycle repeats until the quality criteria are met

What distinguishes AutoDCWorkflow technically is its reliance on the OpenRefine API for cleanup operations. The LLM client generates a sequence of calls to this API in the correct format, thus separating the model's role in planning and inference from OpenRefine's role in actual execution. In our project, the same logic is applied with Pandas instead of OpenRefine because Python is more flexible and compatible .with the rest of the system components

This paper tested three open-source models on the evaluation set. Llama 3.1 and Gemma 2 achieved significant improvements in data quality, exceeding the baseline across all metrics. Gemma 2-27B consistently produced high-quality tables and accurate responses, while Gemma 2-9B excelled at generating a cleanup workflow similar to the human reference version. The key finding is that the models can identify and correct errors by leveraging context within the same row, but they struggle to detect structural errors that .require understanding the distribution across multiple rows

What did the study add to our research

In our project, the DataCleaningTool will apply AutoDCWorkflow logic at three levels. At the selection level, it will focus cleaning on columns relevant to the user's objective. At the inspection level, it will generate a report on the quality of each column before any cleaning. At the decision level, it will choose the method for handling each problem based on the nature of the data: the median for skewed distributions, the mean for normal distributions, and the most frequent value for categorical data. These three layers make cleaning intelligent and justified, rather than a .blind application of fixed rules

2.6 Summary of Reference Studies

These five papers together form a comprehensive research framework covering every component of the proposed client. ReAct provides the theoretical foundation for the inference loop and justifies the transparency of thought. InfiAgent-DABench demonstrates the app's feasibility for data analysis and provides the reference infrastructure for Sandbox, Streamlet, and evaluation. Data Interpreter demonstrates the automation of the entire workflow and inspires the sequence of the five tools. DatawiseAgent provides the latest workflow organization and recall ordering practices and demonstrates effectiveness across diverse models. AutoDCWorkflow deepens the intelligent cleaning logic and justifies DataCleaningTool's decisions in choosing .how to address each problem

Every technical decision in our project is supported by a peer-reviewed research paper. The project does not reinvent the wheel but builds on this work and adds what the existing papers do not offer: handheld tools built with fully customized logic, explanations in natural language for the non-technical user, and evaluation with an .objective comparison with a human data analyst